hashlock.

Security Audit

Frienly (Subgraph - Off-Chain Component)

August 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

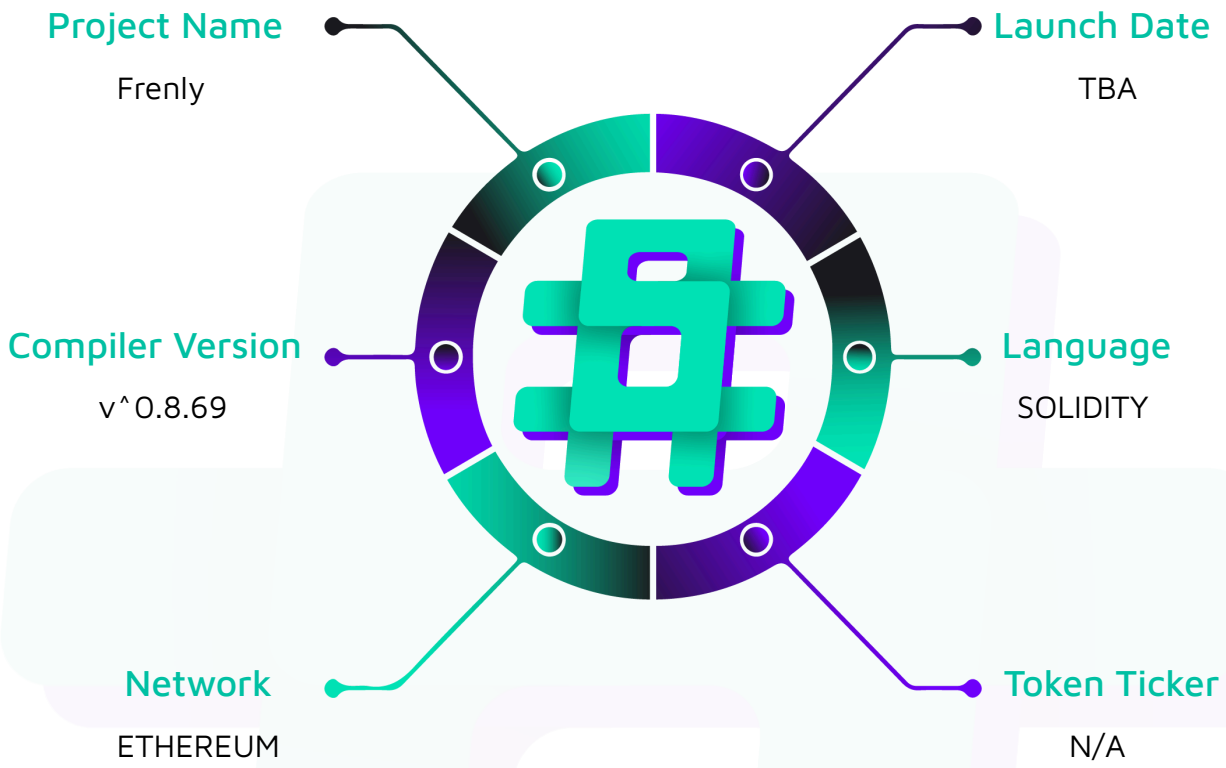
The Frenly team partnered with Hashlock to conduct a security audit of their subgraph component . Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the given components are secure.

Project Context

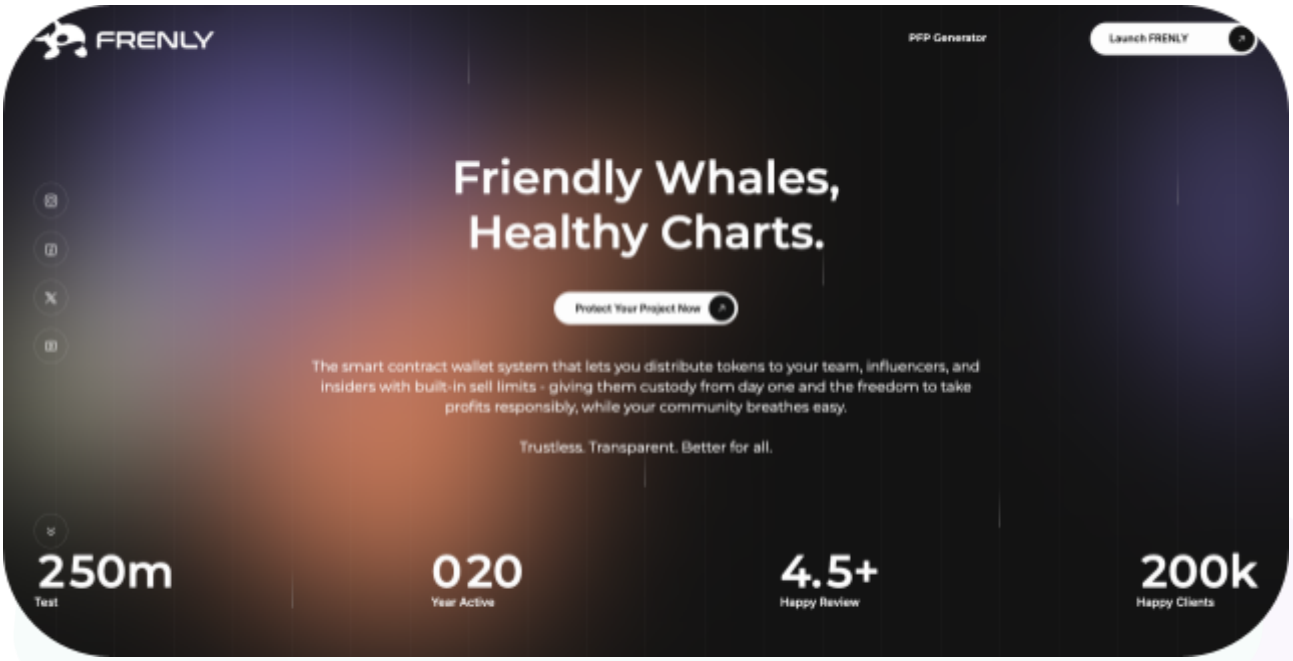
FRENLY is a smart-contract–based wallet solution that helps crypto projects distribute tokens to team members, influencers, or partners in a secure and market-friendly way. It allows projects to maintain custody from day one while enforcing automatic sell limits—both per transaction and per day—to prevent sudden dumps and protect price stability. Compatible with networks like Ethereum, Polygon, and Arbitrum (with Solana coming soon), FRENLY offers flexibility through its built-in DEX and integration with PoolParty for full liquidity when needed, making it an ideal tool for responsible token distribution.

Project Name: Frenly
Project Type: Subgraph - Off-Chain Component
Compiler Version: ^0.8.20
Website: https://www.getfrenly.com/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the Subgraph code within the Frenly project, the scope of work included a comprehensive review of the several files listed below in the given commit. We tested the given scripts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Frenly Subgraph Scripts**
Platform	**Ethereum / Solidity, TypeScript**
Audit Date	**August, 2025**
Audited GitHub Commit Hash	8252bb5bfde90a13625b03b18f2b5ebdd01141ba
Fix Review GitHub Commit Hash	afd059e6f483c2227cbc287c95ed3c0b9321610d

Security Rating

After Hashlock's Audit, we found the scripts to be **"Secure"**. The scripts all follow simple logic, with correct and detailed ordering.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

2 Medium severity vulnerabilities

1 Low severity vulnerability

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This audit scope involves the subgraph scripts of the Frenly project, as outlined in the Audit Scope section. All scripts, test files, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Frenly project subgraph code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the scripts used in this infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Vault-mapping#handleSell - Misinterpretation of Vault Sell event field toDEX as a destination token

Description

The `Sell` event in the Vault contract includes a `toDEX` parameter, which represents the DEX/router address used for the swap, not the destination token. The mapping incorrectly treats `toDEX` as a token, creating a `Token` entity for it and polluting the token catalog with non-token addresses.

Impact

Incorrect analytics for destination tokens, as dashboards will show DEX addresses instead of tokens.

Potential for runtime errors if pricing is attempted on these non-token contracts.

Recommendation

- Remove the logic that creates a `Token` entity from the `toDEX` address in the `handleSell` handler.

- Set `Sell.destinationToken` to null or undefined in this handler, or refactor the schema if the destination token cannot be reliably sourced.

- If the DEX address is useful, store it in a separate field like `dexAddress`.

- Change the Sell Event respectively to include `DestinationToken`.

Status

Resolved

[H-02] Factory-events#handleVaultDetails - Overwriting existing Vaults

Description

The `handleVaultDetails` handler creates a `new Vault(vaultId)` instead of loading the existing entity that was created by `handleVaultCreated`. This occurs as both events are in the same transaction.

Impact

Fields set in `handleVaultCreated` (e.g., `influencer`, `name`) are lost or reset to default values.

The state of the `Vault` entity becomes inconsistent and unreliable.

Recommendation

- Refactor `handleVaultDetails` to first attempt to load the `Vault` entity.

- If the entity exists, update its properties; if it is null, then create a new one.

Status

Resolved

[H-03] Factory-events#handleVaultCreated&handleVaultDetails - Duplicate Frenly creation for Vault

Description

`VaultFrenly.create` is called in both `handleVaultCreated` and `handleVaultDetails`. Since The Graph does not deduplicate Frenly instances by address, this can create two instances for the same vault.

Impact

Risk of double-processing all downstream events from the `Vault` contract.

Data duplication and incorrect aggregations (e.g., `totalSold`).

Recommendation

- Call `VaultFrenly.create` only once, preferably in `handleVaultCreated`.

- Alternatively, add a boolean flag like `FrenlyInitialized` to the `Vault` entity to ensure `create` is only called once.

Status

Resolved

Medium

[M-01] factory-events&vault-mapping#OTCEthClaimed - Double-deduction of vault.unclaimedETH

Description

The `OTCEthClaimed` event is handled in both `factory-events.ts` and `vault-mapping.ts`. As the event is emitted from both the `Factory` and `Vault` contracts in the same transaction, both handlers will run, each decrementing `vault.unclaimedETH`.

Impact

The `unclaimedETH` balance will be understated due to the double-counting.

This can lead to invalid negative balances and corrupt analytics.

Recommendation

- Choose a single handler to be the source of truth for updating `unclaimedETH` (preferably the `vault-mapping` handler).

- Remove the decrement logic from the other handler or disable it entirely in `subgraph.yaml`.

Status

Resolved

[M-02] factory-events&vault-mapping - BigInt underflow risks on counters and pool amounts

Description

Multiple handlers decrement counters (e.g., `Token.heldSupply`, `OTCPool.amount`, `Vault.unclaimedETH`) without first checking if the current value is sufficient. During reorgs or with partial data, this can lead to an attempt to subtract a larger number from a smaller one.

Impact

The subgraph will crash on a WASM trap if an underflow occurs.

If it produces negative values, it corrupts the data state.

Recommendation

- Implement a safe subtraction helper function that clamps the value to zero if the delta is greater than the current value.

- Apply this guard to all counter decrements in the codebase.

Status

Resolved

Low

[L-01] subgraph.yaml - Outdated specVersion

Description

The `subgraph.yaml` manifest uses `specVersion: 0.0.5`, which is an outdated version. - The subgraph may be missing features, bug fixes, and performance improvements included in newer versions.

Recommendation

Upgrade `specVersion` to a more recent, stable version (e.g., `0.0.7` or higher) and confirm compatibility with the mapping API version. Use the current version 1.3.0

Status

Resolved

QA

[Q-01] factory-events&vault-mappings - Missing Zero-Address sanity Checks

Description

Handlers do not validate that address parameters from events (e.g., `vault`, `creator`) are non-zero. A bug in the smart contract could potentially emit an event with a zero address. Creation of junk entities keyed by the zero address, leading to polluted data.

Recommendation

- Add a guard at the beginning of relevant handlers to check for and ignore events with critical zero-address parameters.

Status

Acknowledged

Centralisation

The Frenly project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Frenly project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au